

Homework 3: Baseball Data

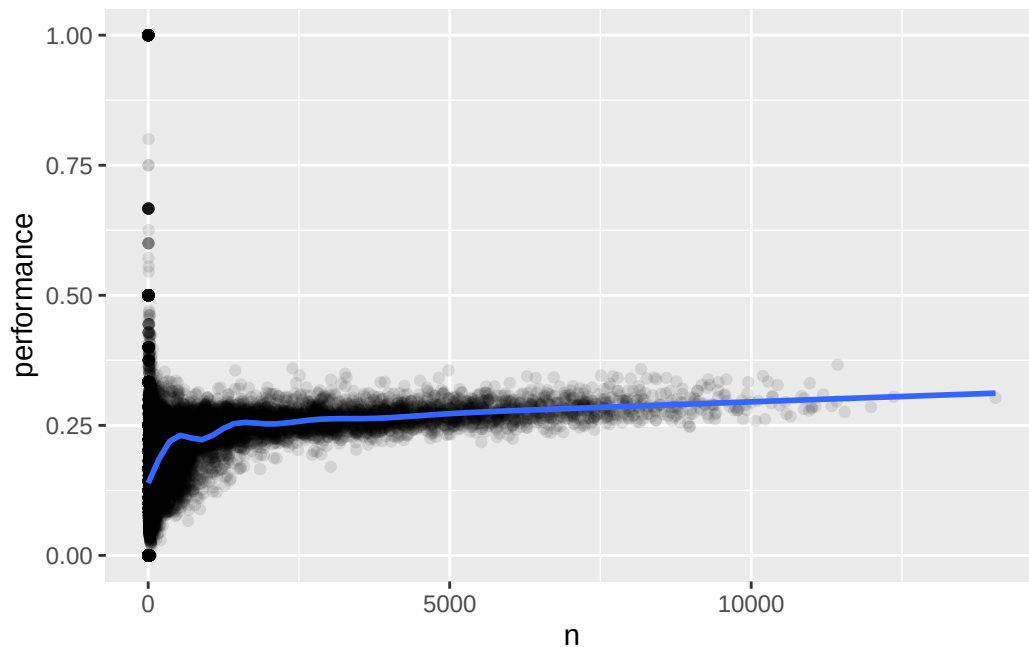
Hadley Stabnau

2026-01-28

1. What line of code did you have to run in R Studio before you could successfully run the code from section 3.6 on your own computer? Why did you need to do that? Do you need to do it again the next time you want to run the code from section 3.6 in your computer?
 - In order to run the code chunk from 3.6 in my own computer, the line of code that need to be added is `library(dplyr)`. This needs to be done because each time the dplyr package is used in a new quarto doc, it needs to be reloaded. This does not need to be done every time I want to run code from 3.6, only when I create a new quarto document.
2. How many variables are in the original **Batting** dataset? Name three or four of them and say what they represent.
 - There are 22 variables in the **Batting** dataset. Some of these variables include: **G** the number of games in which a player played, **HR** homeeruns, **SB** stolen bases and **S0** strikeouts.
3. What variables (columns) are in the **batters** dataset that section 3.6 creates from the **Batting** dataset?
 - In the **batters** dataset that was created in section 3.6, there are 3 variables (columns). These are **playerID**, **performance**, and **n**.
4. What does each row of **batters** represent?
 - Each different row of **batters** represents a different batter based off of their **playerID**.
5. Take the second code chunk in section 3.6 and remove the line that drops rows with n less than or equal to 100. show the resulting code and the new plot you get.

```
library(dplyr)
library(tidyverse)
library(nycflights13)
batters <- Lahman::Batting |>
  group_by(playerID) |>
  summarize(
    performance = sum(H, na.rm = TRUE) / sum(AB, na.rm = TRUE),
    n = sum(AB, na.rm = TRUE)
  )

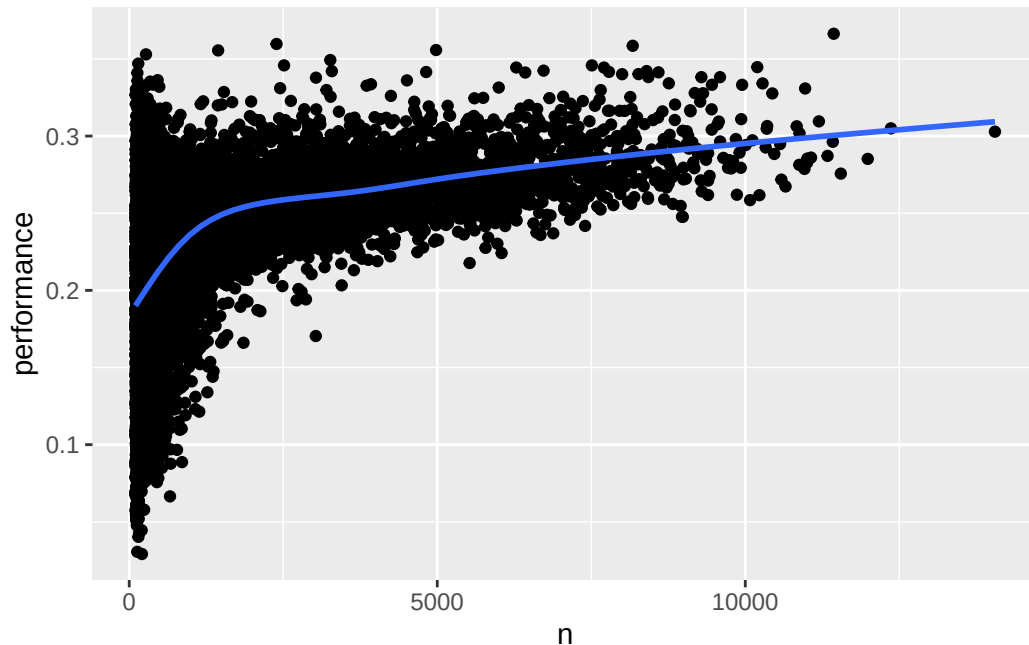
batters |>
  ggplot(aes(x = n, y = performance)) +
  geom_point(alpha = 1 / 10) +
  geom_smooth(se = FALSE)
```



6. Let's figure out what the `alpha` argument does. Modify the second code chunk in Section 3.6 by removing the `alpha` argument so that there is nothing inside the parentheses of `geom_point`. Show the resulting code and the new plot you get. Based on this and on the results of `?alpha`, what does `alpha` control about the plot?

```
batters |>
  filter(n > 100) |>
  ggplot(aes(x = n, y = performance)) +
  geom_point() +
  geom_smooth(se = FALSE)
```

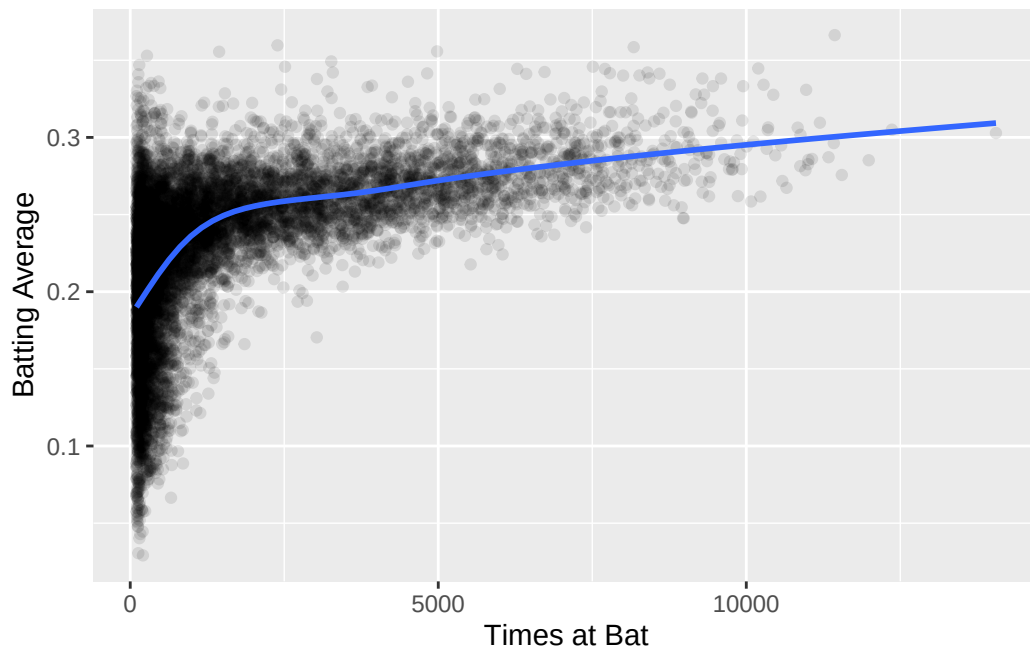
`geom\_smooth()` using method = 'gam' and formula = 'y ~ s(x, bs = "cs")'



- The `alpha` argument changes the level of translucence of the points on the `ggplot()`. A smaller `alpha` value will result in more translucent points, whereas a larger `alpha` value, or one equal to 1 will give the most opaque points.
7. Adjust the second code chunk in Section 3.6 to give the plot more formal and informative axis labels, such as “Batting Average” for the vertical axis and “Times at Bat” on the horizontal axis.

```
batters |>
  filter(n > 100) |>
  ggplot(aes(x = n, y = performance)) +
  geom_point(alpha = 1 / 10) +
  geom_smooth(se = FALSE) +
  labs(x="Times at Bat", y="Batting Average")
```

```
`geom_smooth()` using method = 'gam' and formula = 'y ~ s(x, bs = "cs")'
```



8. Rewrite the first code chunk in Section 3.6 to use the `.by` argument of `summarize` instead of using both the `group_by` and `summarize` functions.

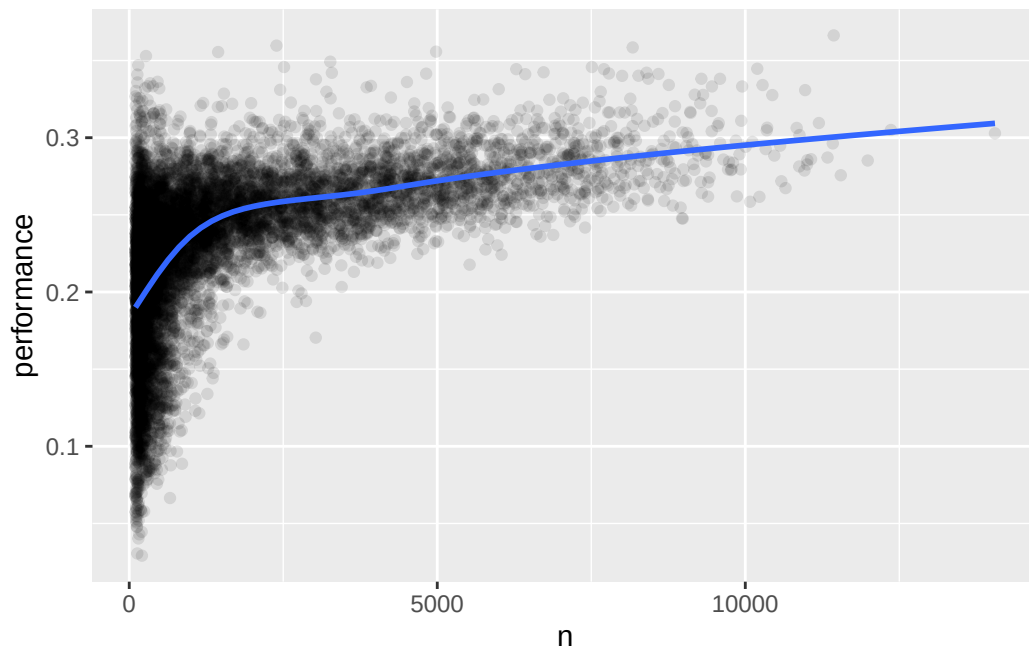
```
batters <- Lahman::Batting |>
  summarize(
    performance = sum(H, na.rm = TRUE) / sum(AB, na.rm = TRUE),
    n = sum(AB, na.rm = TRUE),
    .by=playerID
  )
batters
#> # A tibble: 20,985 × 3
#>   playerID performance      n
#>   <chr>          <dbl> <int>
#> 1 aardsda01      0         4
#> 2 aaronha01     0.305    12364
#> 3 aaronto01     0.229     944
#> 4 aasedo01      0         5
#> 5 abadan01      0.0952     21
#> 6 abadfe01      0.111      9
#> #   20,979 more rows
```

9. Rewrite the second code chunk in Section 3.6 into two separate chunks of code that result in the same final plot. The first code chunk modifies `batters` for plotting by keeping only rows with `n > 100`, and the second chunk plots the new `batters` dataset. As a result of your modified code, the `batters` object itself should now only have values of `n` greater than 100.

```
batters <-batters |>
  filter(n > 100)

ggplot(batters, aes(x = n, y = performance)) +
  geom_point(alpha = 1 / 10) +
  geom_smooth(se = FALSE)
```

``geom_smooth()`` using `method = 'gam'` and `formula = 'y ~ s(x, bs = "cs")'`



10. In the spirit of the first code chunk in Section 3.6, compute a different summary statistic or two related ones from the `Batting` dataset. Briefly summarize in text what you computed. Make sure you update the name of your new dataset from `batters` to something that makes sense for the summary statistic(s) you computed.

```

library(dplyr)
library(Lahman)
Bat <- Batting |>
  group_by(playerID) |>
  summarize(
    performance = sum(H, na.rm = TRUE) / sum(AB, na.rm = TRUE),
    n = sum(AB, na.rm = TRUE),
    avg_hits_per_game = sum(H, na.rm = TRUE) / sum(G, na.rm = TRUE)
  )
batters
#> # A tibble: 20,985 × 3
#>   playerID performance      n
#>   <chr>          <dbl> <int>
#> 1 aardsda01      0         4
#> 2 aaronha01     0.305    12364
#> 3 aaronto01     0.229     944
#> 4 aasedo01      0         5
#> 5 abadan01     0.0952     21
#> 6 abadfe01     0.111      9
#> #   20,979 more rows

```

- I computed the average hits per game for each player by dividing the total number of times they reached base because of a fair ball, by the total number of games played.

11. Read sections 4.22-4.4 of R4DS2e, then apply the style tips there on spaces, line breaks and indentation to improve the style of the code chunk in Section 4.6, Exercise 1.

```

# Section1 -----

library(dplyr)
library(Lahman)
flights|>
  filter(dest=="IAH")|>
  group_by(year,month,day)|>summarize(n=n(),
delay=mean(arr_delay,na.rm=TRUE))|>filter(n>10)

```

`summarise()` has grouped output by 'year', 'month'. You can override using the `groups` argument.

```

# A tibble: 365 x 5
# Groups:   year, month [12]
  year month   day     n delay

```

```

      <int> <int> <int> <int> <dbl>
1  2013      1      1     20 17.8
2  2013      1      2     20  7
3  2013      1      3     19 18.3
4  2013      1      4     20 -3.2
5  2013      1      5     13 20.2
6  2013      1      6     18  9.28
7  2013      1      7     19 -7.74
8  2013      1      8     19  7.79
9  2013      1      9     19 18.1
10 2013      1     10     19  6.68
# i 355 more rows

```

```

# Section2 -----

flights|>
  filter(carrier=="UA",dest%in%c("IAH","HOU"),sched_dep_time>0900,
         sched_arr_time<2000)|>
  group_by(flight)|>summarize(delay=mean(
arr_delay,na.rm=TRUE),
cancelled=sum(is.na(arr_delay)),n=n())|>filter(n>10)

```

```

# A tibble: 74 x 4
  flight delay cancelled     n
  <int> <dbl>      <int> <int>
1     53 12.5         2     18
2    112 14.1         0     14
3    205 -1.71        0     14
4    235 -5.36        0     14
5    255 -9.47        0     15
6    268 38.6         1     15
7    292  6.57        0     21
8    318 10.7         1     20
9    337 20.1         2     21
10   370 17.5         0     11
# i 64 more rows

```